Draw It or Lose It

CS 230 Project Software Design Template

Version 1.0

Table of Contents

CS 230 Project Software Design Template 1

Table of Contents 2
Document Revision History 2
Executive Summary 3
Requirements 3
Design Constraints 3
System Architecture View 3
Domain Model 4
Evaluation 5
Recommendations 6

Document Revision History

Version	Date	Author	Comments
1.0	02/23/2026	Jennifer Joseph	Initial draft of software design document completed.
2.0	02/27/2026	Jennifer Joseph	Reevaluated platforms, compatibility, and dev tools.

Executive Summary

The Gaming Room plans to expand Draw It or Lose It from an Android-only app to a web-based game accessible on any device; this design proposes a centralized server system that ensures unique game and team names, supports multiple teams and players, and maintains one authoritative game instance to provide consistent, scalable, and secure gameplay across platforms.

Requirements

- Support multiple teams within each game
- Support multiple players within each team
- Enforce unique game and team names
- Assign unique IDs to every game, team, and player
- Maintain a single authoritative game instance in memory

Design Constraints

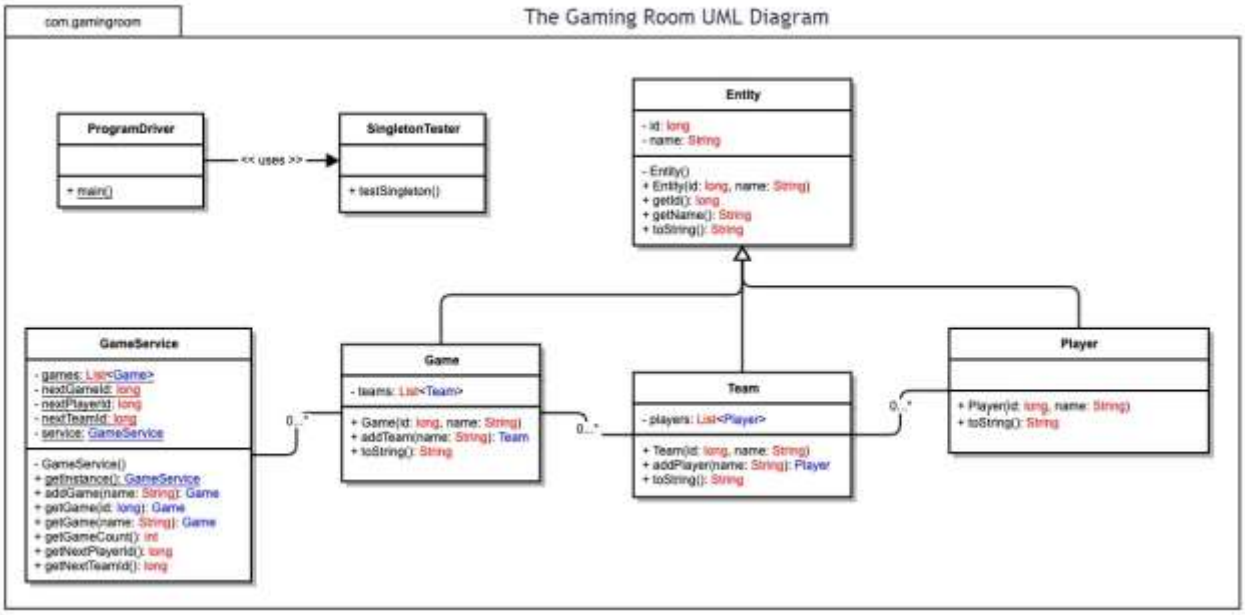
- Handle concurrent users safely by managing name uniqueness and ID generation centrally on the server
- Treat the server as the single source of truth due to network latency and unreliable client connections
- Maintain game state on the server since web communication is stateless but gameplay is stateful
- Design for scalability, recognizing that future multi-server deployments may require session coordination
- Implement strong security controls, including secure communication, server-side validation, and access management

System Architecture View

Please note: There is nothing required here for these projects, but this section serves as a reminder that describing the system and subsystem architecture present in the application, including physical components or tiers, may be required for other projects. A logical topology of the communication and storage aspects is also necessary to understand the overall architecture and should be provided.

Domain Model

The diagram shows a simple hierarchy where a Game contains multiple Teams, and each Team contains multiple Players. A shared base class (Entity) gives Games, Teams, and Players consistent IDs and names, which supports tracking and enforcing uniqueness. A central GameService manages all games and is designed as a single shared instance (Singleton) so there is one authoritative place to create games, assign unique IDs, and check whether game/team names are already taken. Overall, the design keeps responsibilities clear, reduces duplicate code, and directly supports the requirements for multiple teams/players, unique names, and a single in-memory game service.



Evaluation

Reqs.	Mac	Linux	Windows	Mobile Devices
Server Side	Can host UNIX-based web applications, but it is not well suited for large-scale production deployment. It is less common in enterprise cloud environments and requires Apple hardware, increasing cost. While functional for development and testing, it does not offer the same level of scalable, cloud-native infrastructure support as Linux.	Strongest option for hosting a web-based game. It is stable, secure, and widely supported in cloud-native deployment models such as Infrastructure as a Service (IaaS) and Platform as a Service (PaaS). It integrates well with modern web servers and containerized architectures (e.g., Docker), enabling horizontal scaling and high-availability configurations to support thousands of concurrent players. Most distributions have little to no licensing cost, making it the most cost-effective long-term solution.	Supports enterprise web hosting through technologies such as IIS and Active Directory and integrates well with Microsoft-based infrastructure. It can scale effectively but typically involves higher licensing costs and greater system overhead compared to Linux. It is most appropriate for organizations already standardized on Microsoft technologies.	Not designed to function as server hosts. They lack the uptime reliability, scalability, and centralized control required for a distributed web-based system. In this architecture, mobile platforms should only operate as clients that connect to the centralized server.
Client Side	Users access the game through modern web browsers without requiring a native application. Compatibility depends on proper implementation of HTML, CSS, and JavaScript, along with thorough cross-browser testing—especially in Safari—to ensure consistent rendering and stable real-time gameplay.	Access requires only a standards-compliant browser that supports modern web standards and real-time communication (e.g., WebSockets). Since the application is web-based, compatibility depends on standards adherence rather than operating system differences.	Fully supported through major browsers such as Chrome and Edge. Due to its large user base and hardware variability, cross-browser testing and performance optimization are important to maintain consistent gameplay across different system configurations.	Requires responsive web design, touch input handling, and optimization for smaller screens and variable network speeds. Special consideration must be given to mobile browser behavior on Safari iOS to ensure reliable real-time communication and session stability.
Dev Tools	Supports full-stack development using common programming languages such as Java or Node.js for the backend and HTML, CSS, and JavaScript for the frontend. Compatible with major IDEs (e.g., IntelliJ, VS Code) and useful for testing Safari and mobile browser behavior. Hardware costs may be higher, but most development tools have minimal licensing fees.	Provides strong open-source development tools and closely mirrors typical production server environments. Common stacks may include Java (Spring Boot) or Node.js for backend services and standard web technologies for the client. Well suited for containerization, CI/CD pipelines, and DevOps workflows. Most tools are open-source with minimal licensing costs.	Offers comprehensive development environments such as Visual Studio and supports Java, .NET, Node.js, and cross-platform web technologies. Licensing costs may apply depending on the tool editions used. Suitable for both frontend and backend development within enterprise environments.	Development for this project primarily relies on web technologies rather than native mobile languages. However, additional testing tools, emulators, and device testing are required to validate touch interactions, performance, and network resilience. Supporting multiple device types may require coordination between frontend, backend, and QA roles.

Recommendations

1. **Operating Platform:** A cloud-hosted Linux server is the most appropriate operating platform for expanding Draw It or Lose It to additional computing environments. Linux provides strong stability, scalability, and cost efficiency while supporting modern cloud-native deployment models. Because the game must support multiple concurrent sessions and potentially thousands of users, Linux's compatibility with containerized deployment and horizontal scaling makes it a practical long-term solution. Its minimal licensing costs and strong community and enterprise support also make it financially sustainable as the system grows.
2. **System Architecture:** Linux uses a modular monolithic kernel architecture that efficiently manages system resources such as CPU scheduling, memory allocation, and process control. The kernel operates separately from user-space applications, which improves stability and security by isolating processes. This architecture allows multiple game sessions to run simultaneously without interfering with one another. Linux also supports virtualization and containerization, enabling services to run in isolated environments while sharing the host kernel. This makes it well suited for distributed applications where scalability and process isolation are essential.
3. **Storage Management:** To manage storage effectively, static image assets and structured application data should be separated. The approximately 1.6 GB image library can be stored in scalable object storage, while structured data such as players, teams, and game sessions should be stored in a relational database to enforce integrity and uniqueness constraints. Linux file systems such as ext4 or XFS provide journaling capabilities that improve reliability by preventing data corruption. Backup policies, replication, and redundancy should also be implemented to ensure durability and support long-term scalability as content and user activity increase.
4. **Memory Management:** Linux uses virtual memory, paging, and process isolation to efficiently allocate and manage RAM. Virtual memory allows applications to operate as if more memory is available than physically exists, while paging ensures efficient use of available resources. For Draw It or Lose It, memory management is critical because high-definition images must render quickly during timed rounds. By loading only necessary assets into memory and releasing resources when sessions end, the system can maintain responsiveness even with multiple concurrent users. Process isolation further ensures that one session does not negatively impact another.
5. **Distributed Access:** To enable communication across multiple platforms, the application should use a client-server distributed architecture in which the Linux server hosts the core game logic and client devices access it through a web-based interface. Communication can be handled using RESTful APIs for structured data exchange and WebSockets for real-time gameplay updates. This approach supports low-latency communication required for timed image rendering. The architecture must also account for network variability, temporary outages, and dependency between components such as the server, database, and storage systems. Load balancing and redundancy can reduce single points of failure and improve availability. This architecture also allows the application tier to scale independently from the database and storage layers, reducing tight coupling between components and improving fault isolation within the distributed system.
6. **Security:** Security should protect user information both at rest and in transit. The Linux platform supports strong permission controls, firewalls, and access management mechanisms. All client-server communication should be encrypted using HTTPS with TLS to protect data during transmission. Role-based access control can limit administrative privileges, and database-level protections can secure sensitive user information. Regular updates, monitoring, and system hardening practices will further strengthen the platform's ability to protect user data across distributed environments.